Staff permissions: the catalogue codes become real

Report of step 1 on branch feat/staff-permissions and a developer guide for steps 2 to 7, 18 October 2026

Status	Plan approved 18 October 2026 with its six decisions. Step 1 (framework) implemented and tested, not committed. Steps 2 to 7 are described in section 4 as a guide; no code of theirs is written.
Scope	The admin edge derives only ROLE_ADMIN and ROLE_SUPER, so every staff account can moderate announces, release payouts, block conversations and send bulletins. The permission codes of `PermissionCatalogue` exist, the roles screen assigns them, nothing reads them. The task makes them real end to end: edge authorities, edge path rules, framework check, backend routes, seed, SPA, docs.
Step 1 built	`Caller.can(permission)`, `Callers.staff(headers, permission)` answering 403 `missing-permission`, role parsing that keeps permission codes lower-cased, three new unit tests, the rule in doc/cqrs-reactive.md section 5a. Four files changed.
Tests	mvn -pl cqrs-reactive test: 15 classes, 61 tests, 0 failures (CallersTest 7). No other module was rebuilt; the change is additive, and no backend reads role values outside the framework.

1. The problem, as surveyed on 17 October

- **Two authorities only.** `gateway-admin/auth/AdminAccounts` turns an account's roles into `ROLE_ADMIN` (any enabled role) and `ROLE_SUPER` (a role holding `*`). `security/SecurityConfig` needs ROLE_ADMIN on `/api/**` and `/ws/**`, ROLE_SUPER on admins, roles and permissions. `X-User-Roles` carries those two values downstream.
- **Backends check the type, never a permission.** Every staff route in announce-management, order and messaging calls `Callers.staff(headers)` (or `Principals.staff`), which only asks for `X-User-Type=ADMIN`. The roles land in `Caller.roles` and are read nowhere.
- **The catalogue is decorative.** `web/admin/PermissionCatalogue` lists nine codes; the role editor stores them in `announce_permission`; seed roles 2 and 3 still hold litemall's `admin:category:*` strings.
- **Catalog writes are open.** `CategoryController` and `BrandController` have no caller check at all. The customer edge forwards their writes to any signed-in member, so a member can create categories today. The admin edge routes them at `/api/categories/**` and `/api/brands/**`, not under `/api/admin/` as the task block assumed.
- **Role values are upper-cased.** `Callers.roles()` upper-cases every entry of `X-User-Roles`; permission codes are lowercase, so they would never match.
- **Staff see every conversation.** `Principals.viewer` lets any staff account attach to any conversation (REST and WebSocket); the admin edge routes `/api/conversations/**` under plain ROLE_ADMIN.

2. The six decisions (approved 18 October 2026)

#	Decision	Why
1	Catalog rule on the real paths: non-GET on `/api/categories/**` and `/api/brands/**` needs `admin:catalog:manage`; GET stays ROLE_ADMIN. The write methods of the two controllers call `Callers.staff(headers, "admin:catalog:manage")`.	The SPA and the edge use those paths; reads are public on the customer edge anyway; the backend check also closes the member hole.
2	`/api/conversations/**` on the admin edge also needs `admin:support:manage`, and `Principals.viewer` checks that permission when the caller is staff.	Otherwise a staff account without support rights still reads any conversation; viewer is the one staff path of ConversationController and the conversation WebSocket.
3	Order keeps importing the framework `Callers` directly.	Its four roots may share nothing (ContextBoundariesTest); a helper per root would be four copies.
4	Role parsing: values starting with `ROLE_` are upper-cased, everything else lower-cased; `Caller.can` compares lower-cased.	Authorities keep their Spring spelling; codes stay readable.
5	`PermissionCatalogue` moves from `web.admin` to `auth`.	`AdminAccounts` (auth) must read it; auth importing a web package is upside down.

#	Decision	Why
6	One `StaffPermissionRules` table (path patterns, optional methods, permission) builds the `authorizeExchange` rules and is consulted again by the access-denied handler.	Spring's denial carries no reason; the table lets the 403 problem name the permission (`missing-permission`), and keeps today's `forbidden` for anything else.

3. Step 1 as built: the framework rule

All in `cqrs-reactive/src/main/java/org/linlinjava/announce/cqrs/security`. The framework knows no catalogue: a permission is a plain string the admin edge rule, the backend route and the seed roles agree on.

Call	Who passes	Otherwise
`caller.can(permission)`	roles hold the code (lower-cased compare), `*` or `ROLE_SUPER`	false; also false for members, for `ROLE_ADMIN` alone and for a null or blank code
`Callers.staff(headers, permission)`	type ADMIN and `can(permission)`	401 `no-principal` without an id, 403 `not-staff` for a member (the existing checks run first), then 403 `missing-permission` with "This action needs the permission " as detail
`Callers.roles(header)`	comma separated `X-User-Roles`	`ROLE_*` upper-cased, every other value lower-cased, trimmed, empties dropped (`Callers.normalise`)

Code

```
// Caller.java
public static final String ROLE_SUPER = "ROLE_SUPER";
public static final String ALL_PERMISSIONS = "*";

public boolean can(String permission) {
    if (permission == null || permission.isBlank()) return false;
    return roles.contains(ALL_PERMISSIONS) || roles.contains(ROLE_SUPER)
            || roles.contains(permission.trim().toLowerCase(Locale.ROOT));
}

// Callers.java
public static final String MISSING_PERMISSION = "missing-permission";

public static Integer staff(HttpHeaders headers, String permission) {
    Caller caller = any(headers);                       // 401 no-principal
    if (!caller.staff()) throw new CommandRejectedException(FORBIDDEN, NOT_STAFF, "Only staff can do this");
    if (!caller.can(permission))
        throw new CommandRejectedException(FORBIDDEN, MISSING_PERMISSION, "This action needs the permission " + permi
    return caller.id();
}

static String normalise(String value) {               // used by roles(header)
    return value.regionMatches(true, 0, "ROLE_", 0, 5) ? value.toUpperCase(Locale.ROOT) : value.toLowerCase(Locale.RO
}
```

Tests added to `CallersTest` (7 tests in the class now)

- `readsMemberAndStaffWithTheirRoles` also asserts that `"ROLE_ADMIN, Admin:Support:Manage ,admin:dashboard:read"` parses to ROLE_ADMIN, admin:support:manage, admin:dashboard:read (order is not kept: the record copies the set).
- `canHoldsForTheCodeTheStarAndTheSuperAuthorityOnly`: the code, padded and upper-cased input, a missing code, null and blank, ROLE_SUPER, `*`, a member, ROLE_ADMIN alone.
- `staffRoutesWithAPermissionRefuseWhoeverLacksIt`: a support-only caller passes its own code and is refused `missing-permission` (message names the code) for moderation; super and `*` pass; a member is still `not-staff` and nobody still `no-principal`.

Documentation

doc/cqrs-reactive.md section 5a gained a "Permissions (2026-10-18)" bullet: the header content, the normalisation, `Caller.can`, the overload and its 403, the no-catalogue rule, the per-context helpers, and the token-lifetime caveat (a role change reaches a backend at the next access token, no revocation list).

Files changed

```
cqrs-reactive/src/main/java/org/linlinjava/announce/cqrs/security/Caller.java
cqrs-reactive/src/main/java/org/linlinjava/announce/cqrs/security/Callers.java
cqrs-reactive/src/test/java/org/linlinjava/announce/cqrs/security/CallersTest.java
doc/cqrs-reactive.md
```

Compatibility note: the only behavioural change for existing callers is that non-`ROLE_` values of `X-User-Roles` are now lower-cased instead of upper-cased. No service outside the framework reads role values (checked with grep over the five backends and edges), and the admin edge sends only ROLE_ADMIN / ROLE_SUPER until step 2.

4. Developer guide for steps 2 to 7

Each step names the files, what to change, the tests to write or adapt, and the pitfalls seen in the survey. The acceptance lines of the CLAUDE.md block apply unchanged. Build each module together with the framework in one reactor so no stale SNAPSHOT from ~/.m2 is used: `mvn -pl cqrs-reactive,cqrs-reactive-test,<module> -P '!webapp' test`.

The one convention, as a table

Route group (admin edge)	Permission	Backend route(s) that must pass it
`/api/admin/announces/**`	`admin:announce:moderate`	announce-management `AnnounceAdminController` (list, detail, withdraw, expire)
`/api/categories/**`, `/api/brands/**` non-GET	`admin:catalog:manage`	announce-management `CategoryController` and `BrandController` write methods (POST, PATCH, PUT, DELETE); GET stays open
`/api/admin/listings/**`	`admin:listing:manage`	announce-management `ListingAdminController` (import, reindex, inventory, stats)
`/api/admin/users/**`	`admin:member:manage`	edge-hosted `EdgeUserController` (the edge rule is the check)
`/api/admin/regions/**`	`admin:region:manage`	edge-hosted `EdgeRegionController`
`/api/admin/auctions|orders|payouts|bids/**`	`admin:order:manage`	order `AuctionController`, `OrderController`, `PaymentController`, `ShipmentController` admin routes
`/api/admin/conversations/**`, `/api/admin/bulletins/**`, `/api/conversations/**`, `/ws/**`	`admin:support:manage`	messaging `AdminMessagingController`, `StaffWebSocketHandler`, staff path of `ConversationWebSocketHandler` and `ConversationController` through `Principals.viewer`
`/api/admin/dashboard/**`	`admin:dashboard:read`	edge-hosted `EdgeDashboardController`
`/api/admin/admins|roles|permissions/**`	ROLE_SUPER (unchanged)	edge-hosted
anything else under `/api/**`	ROLE_ADMIN (unchanged)	-

Step 2. Edge authorities from the permission rows (gateway-admin)

Files. `auth/AdminAccounts.java`, `auth/PermissionCatalogue.java` (moved from `web/admin`; update the imports in `EdgePermissionController` and `EdgeRoleController`), `auth/AdminPrincipal.java` (javadoc only), `AuthFlowTest`.

Change. In `principalOf` replace `hasAllPermissions(enabledIds)` (a count of `*` rows) by a select of the non-deleted `announce_permission` rows of the enabled roles. Build the authorities in this order: ROLE_ADMIN, ROLE_SUPER when `*` is present, then the codes. `*` expands to every code of the catalogue; a code that `PermissionCatalogue.isKnown` rejects is dropped and logged once per code (keep a `ConcurrentHashMap.newKeySet()` of warned codes in the service). The JWT `roles` claim, `AdminSummary.roles` and `X-User-Roles` need no change: they already serialise `principal.authorities()` as a comma-separated list, so the codes travel with them. Keep the field name `roles` so the SPA's `selectRoles` keeps working.

```
private Mono<Set<String>> permissionCodes(List<Integer> roleIds) {
    LitemallPermissionExample example = new LitemallPermissionExample();
    example.or().andRoleIdIn(roleIds).andDeletedEqualTo(false);
    return permissions.selectByExample(example)
            .map(LitemallPermission::getPermission).collect(Collectors.toSet());
}

Set<String> authoritiesOf(Set<String> codes) {   // package-private, unit-testable without a database
    Set<String> out = new LinkedHashSet<>();
    out.add(AdminPrincipal.ROLE_ADMIN);
    if (codes.contains(ALL_PERMISSIONS)) {
        out.add(AdminPrincipal.ROLE_SUPER);
        PermissionCatalogue.ALL.stream().map(Permission::code)
                .filter(c -> !ALL_PERMISSIONS.equals(c)).forEach(out::add);
```

```
            return out;
        }
        for (String code : codes) {
            if (PermissionCatalogue.isKnown(code)) out.add(code);
            else if (warned.add(code)) log.warn("Ignoring unknown permission code {} on a role", code);
        }
        return out;
    }
```

Tests. A pure unit test of `authoritiesOf` (star expands, unknown dropped, order). In `AuthFlowTest` the throw-away super account must now see every catalogue code in `$.admin.roles` next to ROLE_ADMIN and ROLE_SUPER. `EdgeAdminControllersTest` already proves that unknown codes are refused by the role editor (400), so an unknown code can only come from the database.

Pitfalls. The token grows by about 200 bytes; nothing limits it. A role edit takes effect at the next access token (2 h) or refresh: document it, do not add a revocation list (out of scope).

Step 3. Edge path rules and the 403 problem (gateway-admin)

Files. new `security/StaffPermissionRules.java`, `security/SecurityConfig.java`, `security/ProblemResponses.java` (unchanged, reused), `EdgeRoutesTest`.

Change. `StaffPermissionRules` holds an ordered list of `Rule(String permission, Set<HttpMethod> methods, String... patterns)` with the groups of the table above (methods empty = any; the catalog rule lists POST, PUT, PATCH, DELETE). It offers `Optional<Rule> matching(ServerHttpRequest)` (PathPattern match plus method) and a `ReactiveAuthorizationManager<AuthorizationContext>` per rule that grants when the authentication's authorities contain the code, `*` or ROLE_SUPER. In `SecurityConfig` register the rules between the super paths and the ROLE_ADMIN fallback, one line per group, and make the access-denied handler ask the table which rule matched so the body can carry the code:

```
.pathMatchers(SUPER_PATHS).hasAuthority(ROLE_SUPER)
// one line per group, generated from the table:
.pathMatchers(HttpMethod.POST, "/api/categories/**", "/api/brands/**")
        .access(rules.manager("admin:catalog:manage"))
...
.pathMatchers("/api/admin/announces/**").access(rules.manager("admin:announce:moderate"))
.pathMatchers("/api/**", "/ws/**").hasAuthority(ROLE_ADMIN)

.accessDeniedHandler((exchange, ex) -> rules.matching(exchange.getRequest())
        .map(rule -> ProblemResponses.write(exchange, FORBIDDEN, "missing-permission",
                "This action needs the permission " + rule.permission()))
        .orElseGet(() -> ProblemResponses.write(exchange, FORBIDDEN, "forbidden",
                "This action needs a role you do not have")))
```

A member token (type MEMBER, no staff authorities) must keep answering `forbidden` on the admin API, so test the handler with the rules table only for staff authentications, or accept that a member on a ruled path gets `missing-permission` (either is defensible; pick one and assert it).

Tests. In `EdgeRoutesTest` replace the bare `adminToken()` (ROLE_ADMIN only, which every ruled path will now refuse) by `marketplaceToken()` (role 2 codes) and `supportToken()` (admin:support:manage, admin:dashboard:read), and pick the right one in the existing tests (the release call needs admin:order:manage: use the super token or a third token). Add the acceptance test: support token gets 403 `missing-permission` on `POST /api/admin/announces/{id}/withdraw` with the code in `$.detail`, 200 on `GET /api/admin/conversations` whose proxied request carries `X-User-Roles` containing ROLE_ADMIN and the codes; the super token passes one path of every group (MockWebServer answers 200 to each). Keep the super-only test as is.

Step 4. Backends: every staff route names its permission

Module	Change	Fixture to adapt and test to add
announce-management	`goods.interfaces.rest.Callers.staff(HttpHeaders, String)` delegating to the framework; `AnnounceAdminController` passes admin:announce:moderate (4 call sites), `ListingAdminController` admin:listing:manage, the write methods of `CategoryController` and `BrandController` admin:catalog:manage (add the `@RequestHeader HttpHeaders` parameter they lack). Expose `MISSING_PERMISSION` on the helper.	`AnnounceAdminControllerTest`: the staff client (member 42) gains `.defaultHeader("X-User-Roles", "ROLE_ADMIN,admin:announce:moderate"`; add one test: the same id with `ROLE_ADMIN,admin:support:manage` gets 403 `missing-permission` on withdraw, and with `*` is served. One test for a catalog POST without the code. The listing admin test (if present) gains the header the same way.
order	The four controllers keep importing the framework `Callers` (decision 3) and pass admin:order:manage: `AuctionController` 5 sites, `OrderController` 5, `PaymentController` 1, `ShipmentController` 1.	`OrderModuleEndToEndTest.with(...)` adds `X-User-Roles` for staff; add a second staff id with only admin:support:manage asserting 403 `missing-permission` on `/api/admin/orders/{id}/release` and 200 with the code or `*`.
messaging	`Principals.staff(HttpHeaders, String)` and a staff branch in `Principals.viewer` that calls `Callers.staff(headers, "admin:support:manage")` when `caller.staff()`; `AdminMessagingController` 6 sites, `StaffWebSocketHandler` handshake, `ConversationWebSocketHandler` through viewer.	`MessagingEndToEndTest.identity(...)` adds the roles header for AS_STAFF; add: a staff id with `ROLE_ADMIN` only gets 403 `missing-permission` on the queue, and the `/ws/staff` handshake closes with `NOT_AUTHENTICATED` (the handler already maps `CommandRejectedException` to that close code; keep it, or add a distinct code if the SPA should tell the cases apart).

Pitfalls. Tests of announce-management, order and messaging use the live MySQL plus Testcontainers; messaging's suite stalls while a messaging instance runs against `announce_reactive_db` (it keeps overwriting the checkpoint rows the suite resets). Build order: framework first in the same reactor.

Step 5. Seed: roles 2 and 3 get the real codes (db-reactive)

Files. `db-reactive/sql/gen_seed_en.py` (edit), `announce_data_en.sql` and `announce_reseed_en.sql` (regenerated, never hand-edited), `reseed_en.sh` (no change).

Change. Today `PERMISSION = keep("announce_permission")` copies litemall's rows out of the old seed. Replace it by generated rows with the seed timestamp `T`:

```
PERMS = [(1, "*")] + [(2, c) for c in ("admin:announce:moderate", "admin:catalog:manage", "admin:listing:manage",
         "admin:member:manage", "admin:region:manage", "admin:dashboard:read")] \
      + [(3, c) for c in ("admin:support:manage", "admin:dashboard:read")]
PERMISSION = "INSERT INTO `announce_permission` (`id`, `role_id`, `permission`, `add_time`, `update_time`, `deleted`) VALUES "
    + ",".join(row(i, rid, code, T, T, 0) for i, (rid, code) in enumerate(PERMS, 1)) + ";"
```

In the migration text add, before the final commit, a step that rewrites only roles 2 and 3 (administrators created later keep whatever the role editor stored): `DELETE FROM `announce_permission` WHERE `role_id` IN (2,3);` followed by the same INSERT limited to those rows, and fix the header comment that says permissions are kept. Then run `python3 db-reactive/sql/gen_seed_en.py .` from the repository root and `mvn -pl db-reactive test`. `reseed_en.sh` touches the live database: run it only when the user says so.

Step 6. Admin SPA: nav and routes gated by permission (gateway-admin/src/main/webapp)

Files. `app/config/constants.ts` (a `PERMISSIONS` map), `app/shared/auth/authSlice.ts` (`selectCan(code)` and `can(roles, code)`), `useAuth.ts` (can), new `app/shared/auth/RequirePermission.tsx`, `app/shared/layout/Sidebar.tsx`, `app/routes.tsx`, `app/i18n/{en,da,sv}/common.json`, a new `Sidebar.test.tsx`.

```
// authSlice.ts
export const can = (roles: string[], code: string) =>
  roles.includes('*') || roles.includes(ROLE_SUPER) || roles.includes(code);
export const selectCan = (code: string) => (s: HasAuth) => can(s.auth.admin?.roles ?? [], code);

// RequirePermission.tsx: same shape as RequireRole, prop `permission`, texts common.noPermission.title/detail
// Sidebar.tsx: every entry gets `permission` (admins and roles keep `role: ROLE_SUPER`), filter with
```

```
//   (!e.role || hasRole(e.role)) && (!e.permission || can(e.permission)) && (!e.feature || features[e.feature])
// routes.tsx: wrap each page in <RequirePermission permission={PERMISSIONS.x}> like the super screens today
```

Mapping: dashboard (/) dashboard:read; announces and announce detail announce:moderate; categories and brands catalog:manage; listings listing:manage; regions region:manage; users and user detail member:manage; auctions, orders, order detail, payouts order:manage; support, support thread, bulletins support:manage; admins and roles ROLE_SUPER. A hidden screen on a direct URL renders the no-permission alert. The role editor keeps reading `GET /api/admin/permissions`.

Tests. `Sidebar.test.tsx`: render with a store whose admin has roles `['ROLE_ADMIN','admin:support:manage','admin:dashboard:read']` and the chat and trade features on; expect the support and bulletins links present and the auctions, orders, payouts and announces links absent. Extend `authSlice.test.ts` for `selectCan` (code, star, super). Run `npm test`, `npm run i18n:check` (every new key in all three languages, every `t('common.noPermission.*')` literal present in en) and `npm run lint`.

Step 7. Documentation, manual run, report

- `doc/frontend-edges.md`: a new numbered section "Staff permissions" with the table of this guide, the 2 h token caveat, and the recipe to add a permission (catalogue entry, edge rule line, backend route, seed role, SPA nav and route, i18n). Replace the "Still open" paragraph of section 11 by a pointer to it. Record the manual checklist under section 7 in the format of 17a.
- `doc/cqrs-reactive.md` section 5a is already done (step 1).
- Manual run: start the admin edge, announce-management and messaging (service token on, through the edge) against the live MySQL and docker/cqrs. Log in as admin123 (super), create an administrator with role 3 only, log in as it: the support queue works, the announces screen is hidden from the nav and answers the no-permission page on /announces, and `POST /api/admin/announces/{id}/withdraw` answers 403 `missing-permission`. Delete the account afterwards.
- Run the suites in this order and record the counts: cqrs-reactive, gateway-admin, announce-management, order, messaging, db-reactive; `npm test` and `npm run i18n:check` in gateway-admin.
- Add the session's PDF report to doc/ and commit it with the code when asked; update the status line of the memory file and of the CLAUDE.md block if the block changes.

5. Where this leaves the branch

Four files changed, uncommitted, all in the framework and its guide. Every module still compiles against the framework because the change is additive. The admin edge still forwards only ROLE_ADMIN and ROLE_SUPER, so no behaviour changes until step 2 ships the codes and step 4 names them on the routes; steps 2 and 4 must land together with their test fixtures, or every staff route of a module whose routes name a permission will answer 403 to accounts whose tokens were issued before step 2.